

Edge Lane — Architecture Brief

Version 1 · written 2026-09-05. For the teammate who owns positioning, the edge and the tagline, and who needs to answer feasibility and scalability questions about this project without having built it.

Owner of this file: Aamir. **Regenerate the PDF** with `python -m scripts.md2pdf docs/project/EDGE_LANE_HANDOFF.md`.

0. What this document is, and what it is not

It is: a plain-language grounding in what we have built, how it works, what it can and cannot do, and where the genuinely open ground is. It assumes no RF, no radar, no machine learning background. Every technical term it uses is defined in §4 before it is used anywhere that matters.

It is not: a decision. The edge has not been chosen. §12 lists the four candidates the measurements already support, and the test any new candidate has to pass. Beating those candidates is the job; this document exists so that a new idea can be checked against the architecture in five minutes instead of dying three days later.

How to read it if you are short on time. §1 (the problem), §7 (where we are), §8 (what the architecture forbids) and §9 (where the room is). Those four sections are the working set. The rest is reference you can come back to when a specific question arrives.

The whole project in one paragraph

A radar warning receiver has to watch 18 GHz of spectrum but can only listen to 1 GHz of it at a time — a factor of 18. So it sweeps, on a fixed schedule decided before the mission takes off. That schedule spends exactly as long on an empty band as on the band where a new threat just appeared, because it has no way to know the difference. We built a simulator of that problem out of a real radar dataset, proved the simulator is faithful with four validation checks whose pass criteria were written down before we ran them, measured seven classical scheduling strategies inside it, and are now training a scheduler that learns where to listen from nothing but its own hits and misses.

1. The problem, physically

Picture a corridor with 36 doors. Behind some doors, at some moments, someone is talking. You have one ear and you can hold it against exactly one door at a time, for either a twentieth of a second or a tenth of a second, and then you must choose again. You cannot hear through walls. You get 600 of those choices, and then the exercise ends.

Two things make it hard:

1. **You do not know who is behind which door, or when they will speak.** That is the problem statement's phrase *"in the absence of prior reliable intelligence of emitters and their operating characteristics."* Real systems today lean on a library of known emitters built before the mission. This problem statement is explicitly about the case where that library is wrong or absent.
2. **The talkers are not continuous.** A radar sweeps its beam around like a lighthouse. It illuminates you for a fraction of a second, then points elsewhere for seconds. Miss the window and you wait for the next rotation. Some also hop frequency — the same emitter is behind a different door each time it speaks.

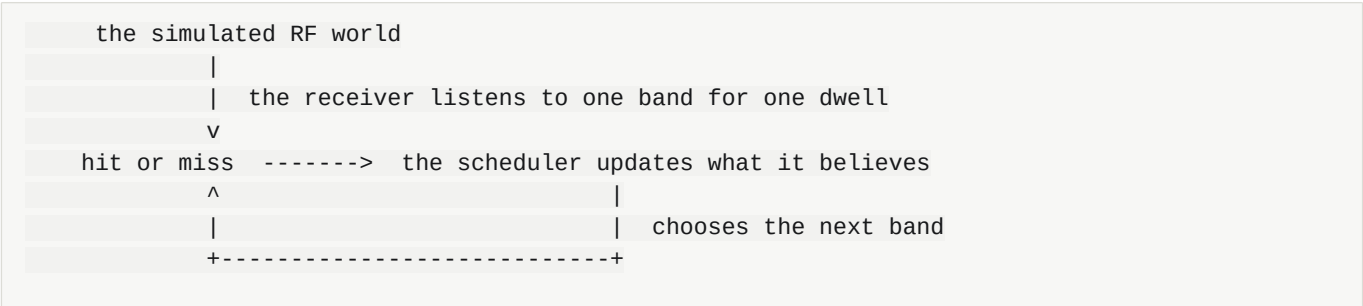
What the current state of the art does. It sweeps in a fixed cycle: door 1, door 2, door 3, and round again. This is called an *open-loop* strategy — the schedule does not depend on anything you have heard. It is predictable, it is fair, and by the problem statement's own words it *"may lose time to nonthreatening emitters by not giving time to new or threatening ones."*

What we are asked to build. The problem statement's Expected Solution, verbatim: "*Machine learning based Electronic Support receiver scheduler software.*" A piece of software that decides which door to listen at next, and gets better at it by being told, each time, hit or miss.

The two things it must do well, and they are named in the problem statement: find emitters *fast* (minimise intercept time) and capture *as much of what they transmit as possible* (maximise interception ratio). §5 explains why those two pull against each other, which turns out to be the most interesting property of the whole problem.

2. The shape of the solution

The whole system is one loop:



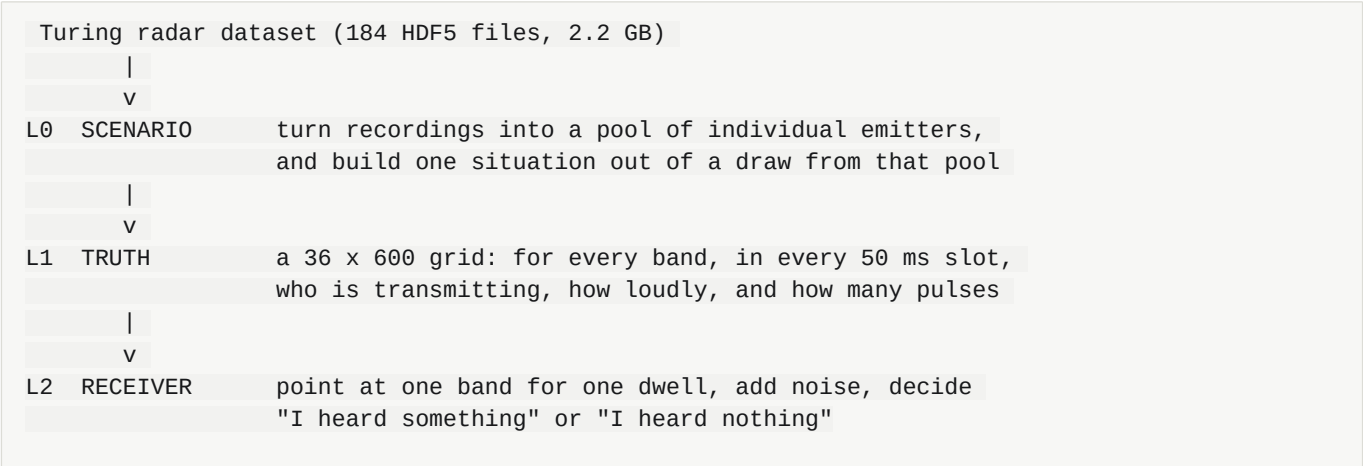
Three components, and it matters that they are separate:

- **The environment** — the simulated world plus the receiver hardware model. Frozen. Identical for every strategy we test. This is what makes a comparison mean anything.
- **The scenario** — one particular RF situation loaded into the environment: which emitters, where, at what frequencies, with what beam behaviour. We have 47 for development and 45 held back, plus the ability to generate new ones.
- **The scheduler** — the decision-maker. This is the only thing we are actually trying to improve. Nine of them exist today (§7); the tenth is the learned one.

The reason for the separation is fairness. Same world, same hardware, different brain — any difference in the score is attributable to the brain. If the receiver model changed between two runs, no comparison between them would be honest.

3. The four layers, and what each hands the next

This is the architecture. Four layers, each simple enough to state in a sentence. Complexity in this project deliberately lives in *validation*, not in the design.



```

      |
      v
L3  AGENT      a standard reinforcement-learning interface:
      36 possible actions, 109 numbers of observation

```

L0 — Scenario. The dataset gives us 92 recorded situations. We do not replay them as-is for training, because a recording answers only "what happened", never "what would have happened if the receiver had looked somewhere else" — and the second question is the entire problem. So we decompose the recordings into individual emitter *contributions* (a pool of 3,443 of them) and build a scenario as a draw from that pool. The dataset placed its emitters independently with no interaction between them, so a recombination is as physically valid as an original — it is the same generative process, one level up, with nothing invented and nothing fitted.

L1 — Truth. A 36-band by 600-slot grid holding, per cell, whether anyone is transmitting, the loudest signal level present, and how many individual pulses arrived. This is the problem statement's own requirement, near-verbatim: *"a simulated RF environment which has truth information on status of emitters in each band and at each time slot."* **Nothing outside the scoring code is ever allowed to read this grid.**

L2 — Receiver. The hardware model. Point at a band, and for each slot of the dwell it measures the true level plus a random noise draw and declares a detection if that clears a fixed threshold. Because of the noise, it will sometimes declare a hit on an empty band (a false alarm) and sometimes miss a real but faint transmission. Those two error rates are the receiver's character, they are measured, and — importantly — **no scheduler can change them.**

L3 — Agent. A standard Gymnasium environment, the same interface every reinforcement-learning library in the world expects. The action is one of 36 bands. The observation is 109 numbers: for each band, how often listening there has paid off, how often we have been there, and how long since we last were — plus how far through the episode we are. All 109 are built purely from the agent's own history. It has no other input.

Scale of the code: eight Python modules, one per layer plus outputs, validation and baselines. **229 automated tests, all passing — re-run 2026-09-05 while writing this document.**

4. The vocabulary

You will hear these constantly. This table is the one to photograph.

Term	What it means
Emitter	A radar transmitting signals. What we are trying to find.
Band	One of the 36 frequency slices the receiver can tune to. Each is 1 GHz wide, centred 500 MHz apart, so neighbouring bands overlap by half.
Slot	50 ms. The clock tick of the whole simulation. An episode is 600 slots = 30 seconds.
Dwell	One act of listening: point at a band, stay there for its fixed length. Seven of the 36 bands take 100 ms (2 slots); the other 29 take 50 ms (1 slot). The lengths are the dataset's own, not ours.
Sweep	One full pass through all 36 bands in order. Takes 2.15 seconds.

Term	What it means
Illumination	One pulse arriving at the receiver. The unit the interception ratio counts.
Intercept	We count an emitter as intercepted when three things hold at once: we were tuned to a band it was transmitting into, <i>its own</i> signal was strong enough to be heard, and the receiver actually declared a detection.
Interception ratio	Of all the pulses that arrived during the episode, what fraction did we capture? Higher is better. One of the two headline metrics.
Censored intercept time	On average, how long after an emitter became detectable did we first find it? Emitters we never found are counted at the full 30 s rather than dropped — that is what "censored" means, and it is essential (§5). Lower is better. The other headline metric.
Coverage	What fraction of the detectable emitters we found at all. Always printed beside the two above.
Pd / Pfa	Probability of detection and probability of false alarm. Receiver properties. Fixed. No scheduler can move them.
Gamma	The detection threshold, −111 dBm. Frozen. Below it the receiver hears nothing.
Rung	One entry on our ladder of scheduling strategies (§7). Rung 1 is random, rung 7 is the learned agent.
Reference line	A strategy that cheats by reading the truth grid. It exists to show what is theoretically available, not to be beaten. Never call one a baseline.
Gate	One of four validation checks the simulator had to pass before we were allowed to quote any scheduler number.
Replay vs sampled	A <i>replay</i> re-runs one recorded situation exactly (used for validation). A <i>sampled</i> scenario is a fresh draw of emitters from the pool (used for training).
Held-out	The 45 scenario pairs nobody has touched. They get used once, at the very end.

5. How the thing is scored — and the conflict at the centre of it

There are three separate families of measurement, and mixing them up is the single most common way to talk nonsense about this project.

Family	The question it answers	Does it change with the scheduler?
Model-level	Is our simulator a faithful model of reality?	No
Receiver-level	How good is the hardware when it looks?	No
Scheduler-level	How well did we aim it?	Yes — this is the only one we can improve

Why this matters to you commercially. If someone asks "so how much did you improve probability of detection?", the honest answer is *"we didn't, and we can't — that is a property of the receiver hardware. What we improved is where it points."* Getting that distinction right in a pitch reads as domain competence. Getting it wrong reads as not knowing what your own numbers mean.

The conflict

The problem statement asks for two things: minimise intercept time, and maximise interception rate. **They pull in opposite directions, and we have measured it on real data.**

A strategy that parks permanently on the single busiest band captures a huge share of all pulses — because that is where most of the pulses are. It also never finds anything else. On our measurements a truth-informed camper reaches an interception ratio of 0.568 against plain round-robin's 0.061 — nine times better — while its average time-to-find collapses from 4.18 seconds to 15.71 seconds and it discovers only 30% of the emitters instead of 87%.

There is a second, subtler version of the same trap. If you measure "average time to find an emitter" over only the emitters you actually found, the camper looks *fast*, because the only things it ever finds are the loud obvious ones. Counting the ones you missed at the full episode length reverses the ranking correctly. This is why the metric is called *censored*, and why the rule in this project is absolute: **never publish an interception ratio without coverage and censored intercept time printed beside it.** A single number hides the entire problem.

The deep version of this, which is candidate edge A (§12): the two objectives sit on opposite sides of an information line. Capturing a pulse does not require the receiver to have *declared* a detection; finding an emitter does. So no single scoring function can be aligned with both, and there is no exchange rate between "a fraction" and "seconds" that any measurement in this project can supply. **That means there is provably no single best scheduler — there is a frontier of them, and which point on it you want is an operational choice, not a technical one.**

6. Why anyone should believe our numbers

This is the strongest and least copyable part of the project, and it is the answer to the consulting question *"how do you know your simulator isn't just telling you what you want to hear?"*

Four validation gates. **Every pass criterion was written into the code before the first run, and there is an automated test asserting that the criteria still match what was written down.** A threshold chosen after you can see the measurement is not a threshold; it is a rationalisation.

Gate	What it does	Result (measured 2026-09-04)
1 — Out-of-sample prediction	Build the simulated world from one set of recordings, then predict what a <i>different</i> set of recordings should contain. Data never used in construction.	accuracy 0.859, MCC 0.685 over 23,594 dwells. Reported, not gated — we deliberately left the pass threshold undecided rather than invent one.
2 — Internal consistency	Rebuild a recording through the pipeline and check it comes back out unchanged, per band.	0.000 percentage points deviation, on every band.
3 — Theory	A controlled artificial case checked against published closed-form mathematics (Köksal), independent of the dataset entirely.	Exact agreement. PASS.
4 — Extremes	The 1-emitter scenario and the 72-emitter scenario, against 12 structural assertions with no tolerances.	12 / 12. PASS.

And we publish what the gates cannot detect. By deliberately injecting faults we established that gate 2's perfect score is algebraically forced and would still pass with a wrong band width, that gate 3's reference shares parameters with the environment, and that **no gate at all covers the receiver's ± 500 MHz band half-width**. We also publish a known limitation: one band is 59% occupied in the recordings and 0% predicted by us, because the recordings we build truth from cannot see that low in frequency.

Naming your own blind spots before a judge finds them converts a weakness into evidence of rigour. Almost no hackathon team validates its simulator at all. None publishes its own failure modes. This is a differentiator on its own, though §12 argues it should sit *under* the headline rather than be it.

7. Where we actually are

Nine strategies have been built and measured against each other on identical scenarios, seeds and hardware settings. **1,539 episodes**, three random seeds, run 2026-09-04.

#	Strategy	Interception ratio	Time to find (s)	Coverage	Beats round-robin on both
1	random	0.0669	4.16	0.858	42.1%
2	round-robin (the open-loop floor)	0.0605	4.18	0.865	—
3	the dataset's own sweep schedule	0.0805	3.74	0.864	51.5%
4	camper, using only what it can see	0.2088	9.67	0.497	1.8%

#	Strategy	Interception ratio	Time to find (s)	Coverage	Beats round-robin on both
5	recency / activity heuristic	0.1104	3.20	0.897	70.2%
6a	published method, ablated	0.1320	4.32	0.860	53.2%
6	published adaptive method (Apfeld)	0.2455	14.86	0.367	4.1%
7	learned scheduler — does not exist yet	—	—	—	—
—	<i>reference line:</i> truth-fed camper	0.5680	15.71	0.296	7.0%
—	<i>reference line:</i> pulse-capture oracle	0.6579	8.01	0.691	19.3%

Reading it: **higher ratio is better, lower time is better, higher coverage is better**. The two bottom rows read the truth grid and are not competitors.

The four things this table says, which are the things worth repeating:

1. **The bar is rung 5, not round-robin.** A one-line rule — go to the band with the best combination of past success and time-since-last-visit — beats the open-loop floor on 70% of individual episodes and beats the dataset's own professional schedule on all three metrics. Claiming victory over round-robin would be claiming victory over a strawman, and a technical judge will spot it instantly.
2. **A camper that cannot see truth is a much weaker camper: 0.209 against 0.568.** The gap of 0.359 is the measurable price of the fact that the thing a good schedule depends on — where the pulses actually are — is invisible to a real receiver. This is candidate edge B.
3. **The published state-of-the-art method's clever half made things worse here.** Its period estimation bought 1.9× the interception ratio for 3.4× the time-to-find, losing to its own ablation. We implemented it faithfully and reported that it lost.
4. **Nothing trivial reaches the good corner.** There is real room between rung 5 and the reference lines, and that room is exactly what rung 7 has to claim.

What is left: the learned scheduler (rung 7), then a single one-shot run on the 45 held-out scenarios that nobody has touched.

8. The boundary — what the architecture restricts

This is the section to read before proposing anything. Each of these is a real constraint with a reason and a cost of violating it. An idea that crosses one of these lines is not impossible, but it is expensive, and the cost should be known before it is pitched, not after.

The frozen ones — crossing these invalidates every number in §7

1. **The scheduler chooses a band. That is its entire vocabulary.** It cannot change how long it listens, cannot change the detection threshold, cannot change gain, bandwidth, antenna or polarisation. Dwell lengths are the dataset's own and are frozen. *Kills:* any edge phrased as "adaptive bandwidth", "adaptive sensitivity", "smart gain control".
2. **The scheduler cannot improve detection quality.** Probability of detection and false alarm are properties of the receiver at a frozen threshold. *Kills:* "our AI improves probability of detection by X%". It is not merely unproven, it is category-incorrect, and saying it will cost credibility with anyone who knows the field.
3. **Retuning is free — we measured it.** Switching bands costs under a millionth of a second, under 0.002% of a slot. Airtime is the only currency in this problem. *Kills:* any edge about minimising retune overhead or settling time.
4. **The environment is frozen, in a file, with a cryptographic tripwire over it.** Band geometry, clock, dwell lengths, noise floor, threshold, the sampling distribution. Changing any of it means re-running all four validation gates and every baseline from scratch. No result from the learned scheduler is allowed to move it.
5. **The held-out data is touched exactly once, at the end.** There is no iterating against it, no "let's just check". Whatever number it produces is the number.

The information ones — these define what "deployable" means here

6. **The agent sees only its own hits and misses.** 109 numbers, all derived from its own scan history. No threat library, no pre-mission intelligence, no carry-over from the previous episode. This is the problem statement's "*absence of prior reliable intelligence*" taken literally. *Kills:* anything that starts "the agent knows that this band usually contains...".
7. **Truth may score the agent; it may never inform it.** Training happens offline and the policy is frozen before it is deployed, so the training-time scoring function is allowed to read the truth grid — it is thrown away before anything ships. The observation is not. This asymmetry is a genuine engineering discipline and is worth 20 seconds of any pitch.
8. **Classification and deinterleaving are not the deliverable.** We are not identifying what kind of radar it is, or separating interleaved pulse trains. The problem statement asks for a scheduler. *Kills:* "and then we classify the threat type" — unless someone decides to expand scope, which is a conversation, not an assumption.

The honesty ones — these are house rules, and four figures have already died to them

9. **Every number must trace to a file and a section, re-checked, never remembered.** This project has already withdrawn four of its own published figures because their measurement convention had not been recorded. Never quote a number from an earlier draft of a deck.
10. **Never quote a headline metric alone.** Ratio, time and coverage travel together.
11. **The two oracle rows are labelled as reference lines everywhere they appear.** Read as competitors they invert the meaning of the entire table.
12. **The dataset is synthetic.** It is a well-constructed synthetic radar dataset from the Alan Turing Institute, named in the problem statement itself, but it is not field data. Any claim about real-world performance is unverified and must be said with that word.

The do-not-claim list

- **No claim of superiority over fielded or classified military systems.** We cannot know what they do, and asserting it is the fastest way to lose a defence-adjacent audience.
- **No real-time hardware performance claims.** We have not measured on target hardware.

- **Nothing from the previous attempt at this project.** Not a number, not a diagram, not a slide. That repository was abandoned precisely because its claims could not be traced.

9. Where the room actually is

Everything in §8 is closed. This is what is open, roughly ordered by how cheap it is to move.

Open surface	Status	Who owns it	Cost
The framing, the story, the tagline	wide open, and it is yours	you	—
The scoring function used during training	three candidates, selection rule fixed in advance	RL lane	days
The learning algorithm and network	entirely unfixed on purpose	RL lane	days
Extra inputs to the agent's observation	open; two specific candidates (angle of arrival, pulse width) are formally undecided	RL lane	~1 day
A preference-conditioned agent — one network, a dial the operator moves at mission time between "find everything fast" and "capture the most signal"	not started; supported by the measured conflict in §5	RL lane	~half a day <i>on top of</i> a working agent
Scenario difficulty and diversity for training	the sampler exists and is frozen; how you <i>use</i> it is not	RL lane	hours
Everything above the environment — operator interface, deployment story, integration narrative, who buys it and why	untouched, and it is where a non-technical lane adds the most	you	—

Read the last row carefully. The technical lane is well ahead of the positioning lane in this project. There is a validated simulator, a measured comparison, four published failure modes and 229 tests — and no answer at all to "who is the customer, what does the integration look like, and what would it take to field this." That gap is your leverage, and nothing in §8 restricts it.

10. Feasibility, in the terms a consultant would ask

Feasibility for this project is **not** "can it be built" — it is built. There are three real feasibility questions and they are different from each other.

10.1 Is the simulator faithful? — largely answered

§6. Four gates, criteria fixed in advance, three pass and one deliberately reported rather than gated. Plus a published account of what the gates cannot see. This is the strongest evidence in the project and it is unusual for a hackathon entry to have any of it.

Residual risk, stated honestly: the dataset is synthetic, one band is not modelled at all, and no gate covers the receiver's band half-width. Say those out loud before someone asks.

10.2 Can the learned scheduler actually run on a real receiver? — argued, not measured

The deployed artefact is a *policy*: a function from "what I have heard so far" to "which band next". It is a small neural network. Three properties make it plausible:

- **No hardware change.** It replaces a schedule, not a component. Same receiver, same antenna, same detector. That is a software upgrade to an existing platform, which is the cheapest kind of defence procurement story there is.
- **The decision budget is 50 milliseconds.** A small network evaluates in microseconds on an ordinary CPU, so there are three orders of magnitude of headroom. **This is an argument from the size of the network, not a measurement on target hardware — do not state it as measured.**
- **It degrades gracefully.** If the policy fails or behaves oddly, the fallback is the existing open-loop sweep, which is what the receiver does today. That is a much easier assurance conversation than a system with no fallback.

Genuinely open and worth your research: what does certification or operational acceptance of a *learned* decision-maker look like in a defence context? Bounded behaviour, explainability, guaranteed worst case, a fallback mode. This is a real question that a defence customer will ask, we do not have an answer, and it is exactly the kind of question your lane is better placed to research than the technical lane is.

10.3 Does it transfer from synthetic data to the real world? — unknown, and say so

Nobody has measured it and nobody in this team can. What we *can* say precisely is that the policy is trained without any prior emitter knowledge whatsoever, so it has nothing dataset-specific memorised to lose — it learns a search behaviour, not a threat library. That is an argument for transferability, and it should be presented as an argument, never as a result.

A strong, defensible framing: the deliverable is a validated environment plus a method. Point it at a different receiver's geometry and a different emitter population and the method is unchanged. That is a more honest and more valuable claim than a performance number that would not survive contact with field data.

11. Scalability, in the terms a consultant would ask

Separate the axes. Some scale for free, some scale with work, and one does not scale at all.

Axis	Today	Does it scale?
Number of bands	36	Yes, structurally. The observation is 3 numbers per band; the action is a choice among bands. Both grow linearly. Discrete choice over a few hundred bands is routine; over many thousands you would need a different action formulation. Unmeasured beyond 36.

Axis	Today	Does it scale?
Spectrum width	250 MHz – 18 GHz	Nothing in the method is tied to those frequencies. It is tied to the <i>ratio</i> between what the receiver can hear at once and what it must cover, which is the actual difficulty knob.
Number of emitters	1 to 82 per scenario, measured	Handled today across that whole range. Denser environments are a training-distribution question, not an architecture question.
Episode length	30 seconds, fixed by the dataset	This one has a known caveat. Our finding that a published method's period estimation hurts is specific to 30-second episodes — a period estimator has little to work with in 30 seconds. On longer missions that result could reverse. Naming this is a strength; hiding it is a liability.
Compute at run time	one small network evaluation per 50 ms	Trivially fine, by argument (§10.2). Unmeasured on target hardware.
Compute at training time	offline, once, on a laptop-class machine	Training is not on the critical path of a deployment.
Multiple receivers cooperating	not modelled at all	This is a genuine extension, not a tweak. Two receivers dividing the spectrum is a different problem — it needs a coordination model the architecture does not have. A legitimate "future work" item; not something to claim.
Real field data	none	See §10.3. Unverified, and the word to use is "unverified".

The honest scalability sentence: *nothing in the method is specific to 36 bands or to this dataset — it is a scheduling policy over whatever band structure a receiver has — but our measurements are all at this configuration, and we say so.*

12. The edge candidates already on the table

Four, each backed by a measured number. **Pick one headline and one supporting. Do not list all four** — a deck that claims four differentiators claims none.

Candidate A — the operator's dial. *No single scheduler is best, because the two things the problem statement asks for provably pull against each other. So we don't ship one scheduler — we ship a frontier, and the operator picks a point on it at mission time.* Backed by the measured conflict in §5 and by the structural result that no single scoring function can serve both objectives. Would land hardest as a live demo with a slider. **Currently the recommended tagline;** needs about half a day of work on top of a working agent.

Candidate B — learning to see what the receiver cannot measure. *The hard part isn't scheduling. It's that the quantity a good schedule depends on is invisible to the receiver — and we can measure exactly how invisible.* The number is the 0.359 gap between the truth-fed camper (0.568) and the same strategy fed only what a real receiver sees (0.209). The claim then becomes "our agent closes X% of that gap", which is a quantity nobody else in the room will have. **Recommended as the technical claim under the tagline.**

Candidate C — we can prove our simulator is right. §6. Criteria fixed before the run, asserted by a test, blind spots published. **Use it as the credibility layer, not the headline** — it wins the "do we believe you" question, which is where most machine-learning pitches quietly lose, but it is not a vision.

Candidate D — what we tried that didn't work. One slide, four bullets: a published method whose clever half made things worse; a paper whose algorithm contradicts its own prose; two of our own baselines that turned out to be the same policy; four of our own figures withdrawn when their measurement convention proved unrecorded. **Most teams cannot fill this slide. It is disproportionately persuasive.**

The test any new candidate must pass

- 1. **It must be a number, not an adjective.** If it cannot be written as a row in the §7 table or a line in a validation report, it is a slogan.
- 2. **Two sentences.** One states it in language someone with no radar background understands. The second is the number that backs it. If the second sentence does not exist yet, name the experiment that would produce it — and if that experiment cannot be run in time, drop the candidate.
- 3. **It must survive §8.** Check the boundary list before you fall in love with it.
- 4. **An edge you have to hedge is worse than one you don't claim.**

Where the best new candidates are likely to come from: the surprises. Every genuine insight in this project so far came from a measurement contradicting the obvious model — the camper that was never reachable, the published method that lost to its own ablation, the two baselines that were secretly one. If you find a fifth surprise, that is probably the edge.

13. How to check anything in this document

Nothing here should be taken on trust, including from me. Every number above comes from one of four places, and all four are in the repository:

To check	Look at
the requirement, verbatim	<code>docs/project/SIH26055_PROBLEM_STATEMENT.md</code>
every metric definition, the ladder table, all four gate results	<code>docs/project/EVALUATION.md</code> §4, §5, §6
why any choice was made, with its evidence and date	<code>docs/project/DECISIONS.md</code> — 47 numbered decisions
the architecture in technical terms	<code>docs/project/PROJECT_ARCHITECTURE.md</code> , <code>docs/project/ENVIRONMENT_SPEC.md</code>
what the RL lane is doing and when it lands	<code>docs/project/RL_LANE_HANDOFF.md</code>
what the deck needs and who owns each slide	<code>docs/project/PPT_LANE_HANDOFF.md</code>

Regenerate the measured results yourself — this takes minutes, not hours:

```
python -m rfenv.validate # the four gates
python -m rfenv.compare --seeds 3 --sampled 10 --figures # the ladder + figures
python -m pytest -q tests/ # the 229 tests
```

The figures live in `runs/baselines/` after that second command: a frontier plot, a band-versus-time picture of each strategy actually working, and a curve of emitters discovered over time. That directory is deliberately not committed to the repository — the figures are rebuilt, never stored, so a picture can never quietly disagree with the table beside it.

Provenance of this document. Every measured figure quoted above is transcribed from `EVALUATION.md` §5 and §6, which record the runs of 2026-09-04 with their commands and dates; the ladder is 1,539 episodes at threshold -111 dBm across 3 seeds. The only measurement re-run while writing this document is the test suite: **229 passed, 2026-09-05**. Where this document and `EVALUATION.md` ever disagree, `EVALUATION.md` wins; where either disagrees with the problem statement, the problem statement wins.

14. What we need back from you

Not deliverables — inputs. These are the things the technical lane cannot produce.

1. **The edge, and the tagline.** One sentence a judge repeats afterwards. Candidates in §12; a better one is welcome and is the point of your lane existing.
2. **Named beneficiary and measurable outcome.** Who specifically is better off, and by what measure. The submission is scored on this and we do not have it.
3. **The integration story.** What does adopting this actually look like for an operator — what changes, what does not, what is the fallback, what has to be certified.
4. **The scope question, answered deliberately:** do we stay a scheduler, or do we frame a wider product? §8 says classification is out of scope *as decided*, not *as impossible*. Widening it is a decision someone can take; it should be taken openly rather than drifting.
5. **Any claim you want to make that §8 forbids** — bring it back rather than dropping it. Some of those lines can be crossed if we choose to pay for it, and knowing which ones you want is more useful than silence.